The Journal of Systems and Software 85 (2012) 1176–1187

Contents lists available at SciVerse ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Controversy Corner

Improving test efficiency through system test prioritization

Hema Srikanth[a,1], Sean Banerjee[b,*]

[a] IBM, Lotus Division, Littleton, MA 01460, United States
[b] Lane Department of Computer Science and Electrical Engineering, West Virginia University, Morgantown, WV 26506, United States

ARTICLE INFO

Article history:
Received 23 January 2011
Received in revised form 29 October 2011
Accepted 2 January 2012
Available online 11 January 2012

Keywords:
Software testing and reliability
Software quality
Value-based testing
System test
Test case prioritization
Regression testing

ABSTRACT

Software testing is an expensive process consuming at least 50% of the total development cost. Among the types of testing, system testing is the most expensive and complex. Companies are frequently faced with budgetary constraints, which may limit their ability to effectively complete testing efforts before delivering a software product. We build upon prior test case prioritization research and present a system-level approach to test case prioritization called Prioritization of Requirements for Test (PORT). PORT prioritizes system test cases based on four factors for each requirement: customer priority, implementation complexity, fault proneness, and requirements volatility. Test cases for requirements with higher priority based upon a weighted average of these factors are executed earlier in system test. An academic feasibility study and three post hoc industrial studies were conducted. Results indicate that PORT can be used to improve the rate of failure detection when compared with a random and operational profile-driven random approach. Furthermore, we investigated the contribution of the prioritization factors towards the improved rate of failure detection and found customer priority was the most significant contributor. Tool support is provided for the PORT scheme which allows for automatic collection of the four factor values and the resultant test case prioritization.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Software testing is a strenuous and expensive process (Beizer, 1990; Craig and Jaskiel, 2002). Research has shown that at least 50% of the total software cost is comprised of testing activities (Harrold, 2000; Tahat et al., 2001). System testing is testing conducted on a complete, integrated system to evaluate the system compliance with its specified requirements (IEEE, 1990). Configuring and testing multiple complete, integrated systems (including hardware, operating system, and cooperating and co-existing applications) that are representative of a subset of customer environments is time intensive. System testing is the last phase before the product is delivered for customer use and thus represents the last opportunity for verifying that the system functions correctly and as desired by customers (House and Newman, 1989). Additionally, companies are often faced with lack of time and resources, which limits their ability to effectively complete testing efforts. Prioritization of the execution order of test cases in a test suite can improve test efficiency (Basanieri et al., 2002; Elbaum et al., 2002; Rothermel et al., 2001, 1999; Srikanth et al., 2009).

Test case prioritization (TCP) involves the explicit prior planning of the execution order of test cases with the intention of increasing the effectiveness of software testing activities by improving the rate of fault detection (Rothermel et al., 2001, 1999; Srikanth et al., 2009). There can be many possible goals behind applying TCP, such as to increase the rate of fault detection; to increase statement, branch or function test coverage; and/or to increase confidence in system reliability (Rothermel et al., 2001, 1999). To date, TCP has been primarily applied to improve regression testing efforts (Elbaum et al., 2002; Rothermel et al., 2001, 1999) of white box, code-level test cases. We extend the white box, code-level TCP techniques and apply TCP at a black box, system-level.

Our research goal is to develop and validate a system-level test case prioritization scheme for identifying the more severe failures earlier in system test. As such, we propose the Prioritization of Requirements for Testing (PORT) (Srikanth, 2004, 2005a; Srikanth and Williams, 2004, 2005; Srikanth et al., 2005a) system-level TCP scheme. PORT prioritizes system-level black box tests by considering four prioritization factors: (1) customer-assigned priority of requirements; (2) developer-perceived implementation complexity; and (3) fault proneness of requirements; and (4) requirements volatility. PORT is designed to be an easy-to-use scheme wherein the factor values can be collected by the engineering team with minimal effort. The open source support Requirements-Based Testing[2] (ReBaTe) tool,

* Corresponding author. Tel.: +1 304 293 3486.
E-mail addresses: srikanth_h@us.ibm.com (H. Srikanth), sean.banerjee@mail.wvu.edu (S. Banerjee).
[1] Tel.: +1 919 389 9488.

[2] http://sourceforge.net/projects/rebate/.

0164-1212/$ – see front matter © 2012 Elsevier Inc. All rights reserved.
doi:10.1016/j.jss.2012.01.007

automates the ordering of system test cases according to the PORT scheme. ReBaTe also automatically provides traceability between tests, requirements, and defects; updates fault proneness as failures are reported; and updates requirements volatility as requirements are changed.

Via our prioritization scheme, we aim to increase test efficiency by improving the rate of failure[3] detection. Test efficiency is a measure of the percentage of failures detected over the period of test suite execution. Through our research, we set out to evolve four theories based upon four factors we believe to be important to system test case prioritization:

T1. ***Customer-assigned priority*** is a measure of the importance of a requirement to the customer. The customer assigns a value for each requirement ranging from 1 to 10 where 10 is the requirement with the highest customer priority.

T2. ***Developer-perceived implementation complexity*** is a subjective measure of how difficult the implementation of a requirement is perceived to be by the development team. Each requirement is analyzed to assess the anticipated implementation complexity and is assigned a value ranging from 1 to 10; the larger value indicates higher complexity.

T3. ***Fault proneness*** allows the development team to identify the requirements which have had the most failures in the previous release.

T4. ***Requirements volatility*** is based on the number of times a requirement has been changed in the development cycle normalized to a range of 0–10.

We examined the contribution of each of these factors toward the prioritization of system test cases to evolve our four theories. One academic feasibility study and three post hoc industrial case studies were conducted by evaluating the testing efforts of software development teams at Tekelec[4] and I-Cubed[5]. Tekelec is a leading developer of switching and signaling telecommunications solutions, and I-Cubed is a provider of content management and digital rights management for computer-aided design (CAD) documents.

In our prior work (Srikanth, 2004, 2005a; Srikanth and Williams, 2005; Srikanth et al., 2005a) we evaluate the benefits of three factors in PORT (v1.0) prioritization namely CP, IC, and RV, and evaluated PORT on four student projects. The contributions of this paper include assessing the benefits of PORT (v 2.0) where we measure the benefits of all four prioritization factors and applied PORT to three industrial projects.

The rest of this paper is structured as follows. Background work and the PORT scheme are discussed in Sections 2 and 3. An overview of a PORT feasibility study is discussed in Section 4. The research methods applied and the results of industrial case studies are summarized in Sections 5 and 6. Finally, a summary is provided in Section 7.

2. Related work

This section provides an overview of the background work.

2.1. Coverage-based test case prioritization

Coverage-based TCP techniques (Elbaum et al., 2002; Rothermel et al., 2001, 1999) involve ranking test cases based on the statement coverage they provide. Test cases are ranked based on the number of statements executed/covered by the test case such that the more lines of code the test executes, the higher the test coverage. For branch and function coverage techniques, tests are prioritized based on the program branches or program functions covered, respectively. Research has shown that prioritization of system configurable software driven not only by fault detection but also by the cost of configuration and setup time improves early fault detection rate (Srikanth et al., 2009).

The benefits of code coverage-based TCP strategies were measured using the Average Percentage of Faults Detected (APFD) (Rothermel et al., 2001, 1999) metric. The APFD value is a measure of how quickly the faults are identified for a given test suite. The APFD values range from 0 to 100 and are monitored during test suite execution. The APFD values represent the area under the curve in a plot of percentage of faults detected (y-axis) against percentage of test suite run (x-axis) of a graph. Suppose we have a scenario with 10 faults labeled 1 through 10 and five test cases labeled A through E. We have 120 possible ways of ordering the five test cases. Table 1 illustrates the faults detected by each test suite. If we choose to execute the tests in the order C-E-B-A-D then the APFD score would be 84%, as determined by the area under the curve in Table 1 (Rothermel et al., 2001, 1999). We analyze a similar validation metric to assess the efficacy of PORT, Weighted Percentage of Failures Detected (WPFD), as will be discussed in Section 4.

If all faults are not equally severe, severity-neutral TCP strategies and the associated APFD metric can provide misleading information (Elbaum et al., 2001). As a result, Elbaum et al. incorporate fault severity in a cost-cognizant Average Percentage of Faults Detected ($APFD_C$) TCP strategy (Elbaum et al., 2001). Instead of representing Test Suite Fraction on the x-axis (as done for APFD), Percentage of Total Test Case Cost Incurred is considered in $APFD_C$. Additionally, instead of representing Percent Faults Detected on the y-axis (as for APFD), Percentage Total Fault Severity Detected is considered in $APFD_C$ (Elbaum et al., 2001). $APFD_C$ gives a higher reward to those test case orderings that reveal the most severe faults earlier. For example, from Table 1 we find that the ordering C-E-B-A-D gives an APFD score of 84% while E-C-B-A-D gives a score of 76%. However, if faults 1–10 do not have the same severity an ordering that yields a better APFD score may not yield a better $APFD_C$ score. For instance, if test cases A, B, D and E took 1 h to complete and test C took 10 h to complete then C-E-B-A-D would yield a lower $APFD_C$ score as we are running a higher cost test (C) earlier in the cycle as evidenced in Table 1. $APFD_C$ can be used to assess the prioritization orders post hoc, i.e. when the severity and cost values are known, and cannot be used for predicting cost and severity values. In the validation of the $APFD_C$ metric, Elbaum et al. assign six levels of severity to the faults in the program. We follow a similar approach of assigning severity to the failures as discussed in Section 3.

2.2. Software Reliability Engineered Test (SRET)

Software Reliability Engineered Testing (SRET) (Musa, 1999, 1996) is a system test approach that involves testing software systems guided by reliability objectives, and criticality of the system operations with the main goal of achieving desired software reliability while spending minimum cost. We discuss below the SRET approach and how we compare our research work with SRET.

Test planning using SRET involves estimating the relative customer use of the functions in a software product via an operational profile. An operational profile is a set of operations and their probabilities of occurrence (Musa, 1999, 1996) where an operation is a task that the system performs. Operational profile estimation is done collectively by testers, system engineers, architects, and

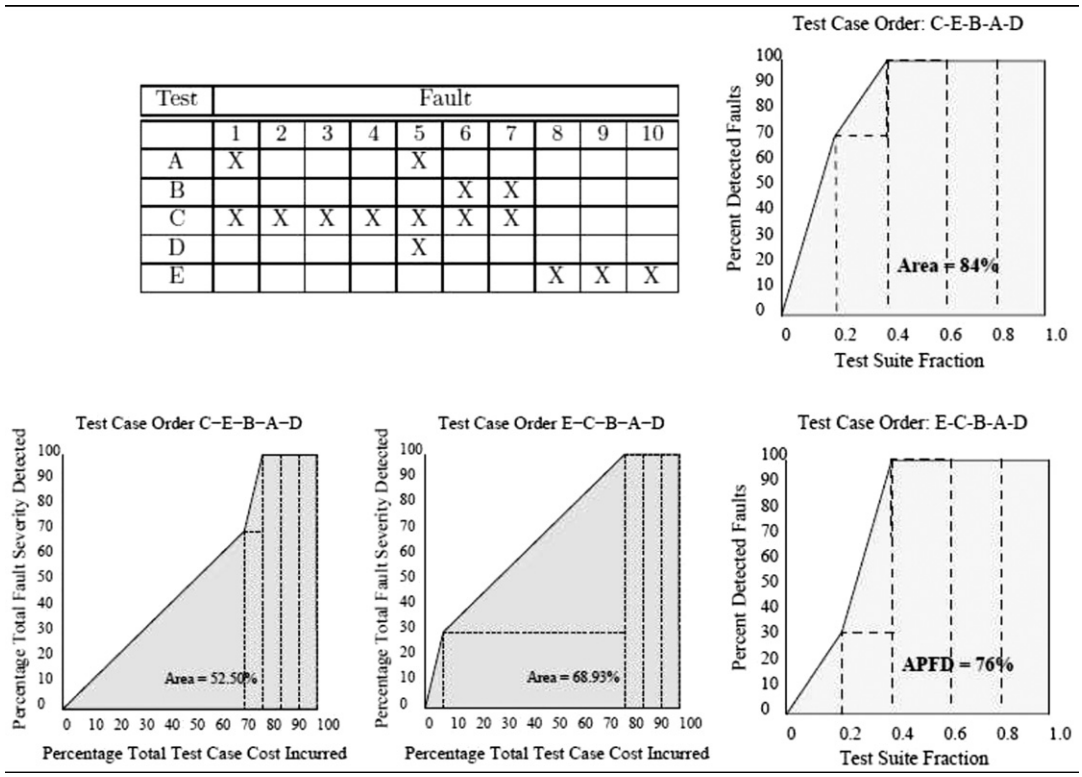
[3] A fault is an *incorrect step, process, or data definition in a program* (IEEE, 1990); a failure is *the inability of a system or component to perform its required function within the specified performance requirement* (IEEE, 1990). Therefore, the purpose of current TCP approaches, which are white box-based is to increase *fault* detection, and the purpose of our system-level, black box TCP approach is to increase *failure* detection.

[4] http://tekelec.com/.

[5] http://i-cubed.com/.

Table 1
Test suite with faults (Rothermel et al., 2001, 1999).

Test	Fault									
	1	2	3	4	5	6	7	8	9	10
A	X				X					
B						X	X			
C	X	X	X	X	X	X	X			
D					X					
E								X	X	X



customers. Once the operational profile has been estimated, testers determine the total number of test cases that can be written and executed according to budgetary and resource constraints. Then, the number of test cases written for each operation is a relative proportion of the total number of test cases based upon the use percentage for that operation in the operational profile. The SRET approach is an important consideration for test prioritization, utilization of an operational profile allows for the allocation of tests based on the proportion of usage. This prevents the allocation of excessive testing resources to seldom used operations. SRET is also the only known operational profile based approach at the system test level. Once the tests have been written, they are executed in a random order so that reliability growth can be tracked via system test progress.

SRET has been applied to many projects, mostly industrial, for over a period of 18 years. One representative project at AT&T, International Definity, showed that the use of SRET in one product release resulted in increased reliability and customer satisfaction when compared with the previous release of the same product. Additionally, product sales increased by a factor of 10, system costs decreased by a factor of two, total project development time decreased by 30%, and maintenance costs reduced by a factor of 10 (Musa, 1996). Another project at Motorola showed that applying SRET by directing test effort on major functional areas of use by customers reduced the number of failures found in the field (Donovan et al., 2001), thereby increasing test effectiveness. Despite the positive business results from applying SRET, some consider the creation and maintenance of the operational profile to be resource intensive and propose non-operational (Vouk and Rivers, 2003) techniques. We use a similar, however non-operational, approach to SRET in test planning in that we consider customer-assigned priority in our prioritization scheme. Customer priority may be analogous to the operational profile for high priority requirements to the customer.

2.3. Model based test prioritization

Model based test prioritization has been proposed an alternate to traditional code based test prioritization methods. One advantage of a model based approach is the low cost of executing the model as opposed to the actual system. However, the dependence on testers/developers to provide accurate information is one drawback of a model based approach (Korel and Koutsogiannakis, 2009).

In model based test prioritization (Korel et al., 2007, 2005, 2002) system models are used to prioritize test cases. Several modeling languages such as State Charts, Extended Finite State Machine (EFSM), and Specification Description Language (SDL) have been developed to model state based software systems. Model based test prioritization may be difficult to implement in state-less systems, the PORT approach can be applied to both a stateful and a state-less system. At the crux, model based test prioritization utilizes the notion that changes in specifications will lead to changes in the model. Therefore, the differences in the models can be utilized to prioritize the test suite. At the onset, the modified model is executed for the complete test suite and information is garnered on any modifications, prioritizations are then made based on the information collected. The end goal of such an approach is to detect faults in the modified system. In the work in Korel et al. (2005) the authors utilize EFSM for model based test prioritization. In this approach, two models are created to represent the system – one for the original system and one for the modified system. The difference between the models are identified by two sets – one constituting the additions and one constituting the deletions made to transitions in the original model to obtain the modified model. One prioritization approach may be selective prioritization, whereby a test is given a higher priority if it invokes a newly added transition.

In Fraser and Wotawa (2007) the authors apply a model based prioritization technique to various traditional prioritization techniques such as Total Coverage Prioritization, Additional Coverage

Prioritization, Total FEP Prioritization, etc. For instance in Coverage Prioritization a trap property is used and the test cases prioritized by the number of traps they cover. Additionally, in their approach test cases are re-formulated as models and allows for prioritization during the generation of test cases. At the onset each test case is provided a value of importance, initially set to 1. If another test case is found to be a prefix or equal then its importance value is incremented. If a test case subsumes another then its importance value is 1 plus the value of the subsumed test case. In this manner redundant test cases are discarded and the remaining test cases prioritized.

3. Prioritization of Requirements for Test (PORT)

This section presents the four PORT factors and the reasoning behind the factor selection. Also, the factor collection process and the algorithms used to prioritize test cases using the PORT scheme is discussed. The value assigned to all four factors ranges from 0 (low) to 10 (high). Two of the chosen factors – customer assigned priority (CP) and developer perceived implementation complexity (IC) were also suggested in the work by Park et al. (1999). In Park et al. (1999) a voting scheme was used whereby each stakeholder assigned a difficulty (cost) and importance score. A bin model was then used to partition the voted items to create the prioritizations. In our approach we use two additional factors – fault proneness of requirements (FP) and requirements volatility (RV) to prioritize each test case. The usefulness of these additional factors are shown in Section 6.2.

3.1. PORT factors

Each of the four factors will now be discussed.

3.1.1. Customer-assigned priority (CP)

Customer-assigned priority (CP) is a measure of the importance of a requirement to the customer. The customer assigns a value for each requirement ranging from 1 to 10 where 10 is the requirement with the highest customer priority.

Reasoning:

A fault that lies along the path of normal execution results in frequent failures, and the majority of the effort should be spent in finding these faults (Munson and Elbaum, 1999; Musa, 1999). A focus on customer requirements for development has been shown to improve customer-perceived value and satisfaction (Boehm, 2003; Boehm and Huang, 2003; Karlsson and Ryan, 1997). By identifying and more thoroughly testing the fraction of requirements that would be of highest importance to the customer earlier in testing, we aim to increase the business value generated to the customer. The requirements of higher value to the customer should be tested earlier and more thoroughly, which may be important when testing efforts are cut short due to schedule pressures.

3.1.2. Developer perceived implementation complexity (IC)

Developer-perceived implementation complexity (IC) is a subjective measure of how difficult the implementation of a requirement is perceived to be by the development team. Each requirement is analyzed to assess the anticipated implementation complexity and is assigned a value ranging from 1 to 10; the larger value indicates higher complexity. IC is a prioritization factor for requirements being implemented for the first time or for all requirements in the first release. Given the subjective nature of IC, McCabe complexity can be used in instances when the organization has release cycle data readily available.

Reasoning:

Several studies indicate that requirements with high complexity (examined post hoc) tend to have a higher number of faults. Amland (1999) conducted a case study to find that the functions with a high number of faults were the functions with higher McCabe complexity (Amland, 1999). Twenty percent of the system modules result in 80% of the faults (Amland, 1999; Shull et al., 2002; Srikanth and Williams, 2005; Wong et al., 2000), and roughly 50% of the modules are defect free (Shull et al., 2002). To date, no studies have related pre-development developer-perceived complexity and fault proneness.

3.1.3. Fault proneness of requirements (FP)

Fault proneness of requirements (FP) allows the development team to identify the requirements which have had the most failures in the previous release. As the system evolves into several versions, the developers can use the data collected from prior versions to identify requirements that are likely to be error prone. FP is based on the number of field failures and in-house system test defects found in the code that implements a requirement. FP is not considered for new requirements, only for those requirements that have already been in a released product. Let the ith requirement have a suite of test cases $T_i = \{t\}$ which test the requirement i. Let the set of defects $D_t = \{d\}$ be the defects discovered by executing a test suite t. Let there be an additional set of defects $D_i = \{d\}$ that represent the field failures reported against requirement i. This additional set of defects is needed since field-reported failures do not have an associated test case. Finally each defect d has a severity value V_d that is an exponential function of the defect severity. More specifically each defect has a severity of 1, 2, 3, or 4 with corresponding severity values of 2^4, 2^3, 2^2, and 2^1. Studies indicate an exponential increase in cost as severe defects are found later in the product lifecycle, but a much more linear increase for less severe defects (Shull et al., 2002). Our severity values demonstrate a similar pattern.

The fault proneness of the ith requirement, FP_i, is directly related to the number of the failures and the severity of the reported failures for that requirement. The FP values are normalized from a scale of 0–10 as shown in Eq. (1). For example, requirement R_i has had three field failures in the prior release and three system test failures. For simplicity reasons, let us assume both field and in-house failures were Severity 4 defects. S_i (Severity value = 6*2) for requirement R_i is 12. Suppose a requirement R_t has the highest S value such that S_t is 20. Then fault proneness for requirement R_i is (12/20)*10, which is 6.

$$S_i = \sum_{\forall t \in Ti} \left(\sum_{\forall d \in Dt} V_d \right) + \sum_{\forall d \in Di} V_d \qquad \mathrm{FP}_i = \frac{S_i}{\max_{\forall i} \{S_i\}} \times 10 \tag{1}$$

Reasoning:

Ostrand et al. has shown that test efficiency can be improved by focusing on the functionalities that are likely to contain a higher number of faults (Ostrand et al., 2004).

3.1.4. Requirements volatility (RV)

Requirements volatility (RV) is based on the number of times a requirement has been changed in the development cycle normalized to a range of 0–10. Suppose requirement i has E_i recorded changes. The requirement volatility for the ith requirement, RV_i, is computed as shown in Eq. (2). The RV for requirement i is normalized to 10 by dividing the number of edits for requirement i to the highest number of edit for any requirement amongst all the project requirements as shown in Eq. (2).

$$\mathrm{RV}_i = \frac{R_i}{\max_{\forall i} \{R_i\}} \times 10 \tag{2}$$

Suppose requirement R_i, had been edited 15 times since the origin of the requirement. Suppose we have requirement R_t, which has

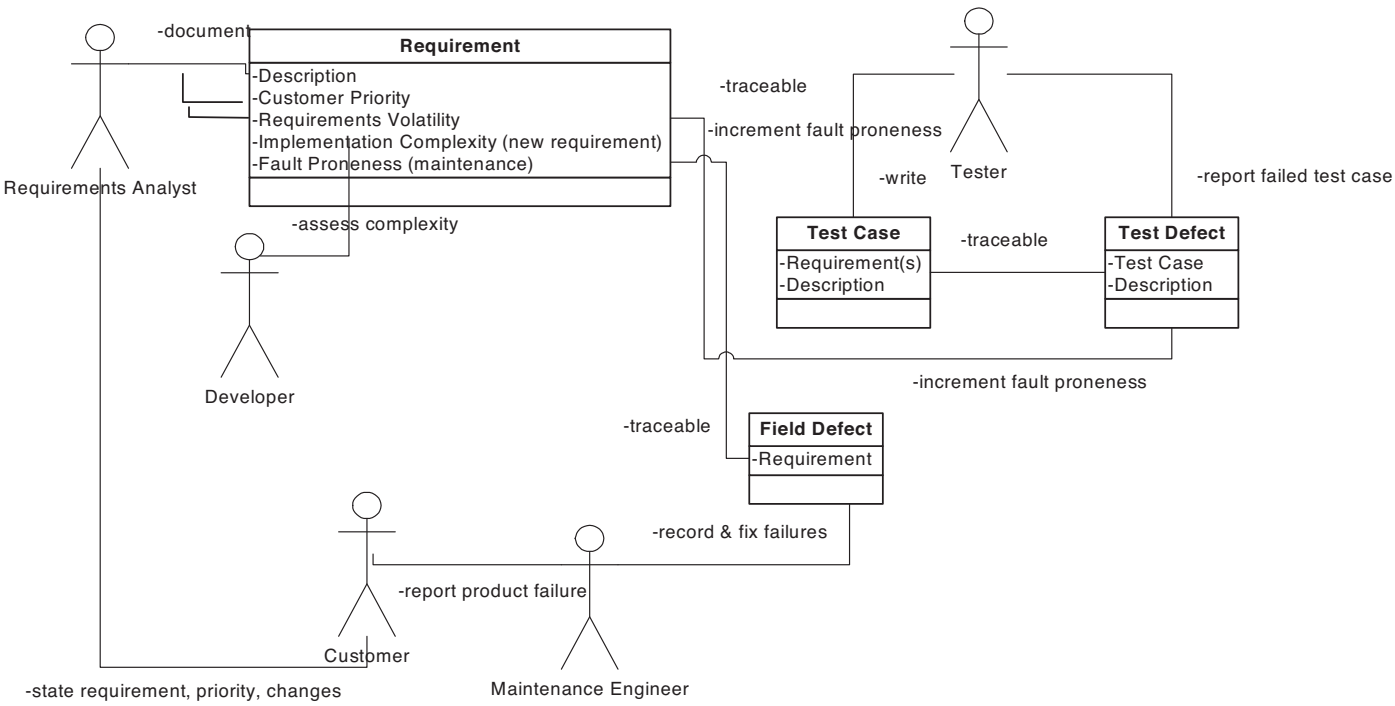


Fig. 1. PORT prioritization factor collection process.

the maximum number of edits amongst all project requirements, say 20. Then RV_i, is equal to (15/20)*10, which is 7.5, and RV_t, is equal to 10.

Reasoning:

Roughly 50% of all faults identified in a project are errors introduced in the requirements phase (Mogyorodi, 2001). On average, severe defects that escape into the field can cost 100 times more to fix after delivery than correcting the same problem in the requirements phase (Shull et al., 2002). Studies conducted by the Standish Group report that 30% of all projects are canceled before completion, and 70% of the remaining projects fail to deliver the required system functionality. The most significant factor to cause these project failures was changing requirements (Standish.Group, 1995). Other studies show the cause for project failures to be lack of user input, and changing or incomplete requirements (Jones, 1996; Mogyorodi, 2001; O'Neal and Carver, 2001). Roughly 25% of the requirements for an average project change before project completion (O'Neal and Carver, 2001), and volatile requirements tend to make the testing activities difficult and cause the software to contain high defect density (Malaiya and Denton, 1999). Changing requirements result in re-design, and often an increase in the fault density in the program (Malaiya and Denton, 1999; Srikanth, 2004, 2005a; Srikanth and Williams, 2004).

3.2. PORT prioritization factor collection process

The process for collection and updating the PORT factors is shown in Fig. 1. The four factors required for PORT are generated as follows: the customer provides the customer assigned priority (CP) to the requirements. The development team takes such requirements and provides a measure of how complex they are, thereby generating the developer perceived implementation complexity (IC). The development team also determines FP of each requirement by assessing the field failures of prior versions. Finally, the customer and requirement analyst together are responsible for how often the requirements change, in the instance of PORT they provide the metric for requirements volatility (RV).

There are five stakeholders in the process; the stakeholder roles are defined below:

- The ***customer*** states the system requirements, the priority for the each requirement, and any additions or changes to the requirements throughout development. The customer also reports field failures to maintenance engineers after product delivery.
- The ***requirements analyst*** records the requirements and associated priorities and any changes to requirements.
- The***maintenance engineer*** fixes defects when field failures are reported and maps the failures back to the requirement(s) that were impacted by the failure.
- The ***developer/architect*** provides a subjective assessment of how complex a requirement is to implement.
- The ***tester*** writes test cases for each requirement, mapping the requirement to its test case(s), and runs the test cases. Test case failures are reported, mapping the test case failure to the test case that revealed the failure.

The open source ReBaTe tool displays the PORT-computed ordering of system test cases. ReBaTe automatically increments requirements volatility when a requirement is changed and increments fault proneness when failure is reported. In the absence of a tool, these values can be tracked manually. Fig. 2 presents a flow chart to represent the PORT structure.

The four factor values are used to compute the Prioritization Factor Value (PFV) of the requirement, which ultimately is used to produce a ranked list of test cases. Factor values are assigned during the design analysis phase, and evolve continually during the software process. Additionally, a factor weight is assigned to the four factors. Factor weight, which is unique for each project, allows the PORT user to customize the priority of each factor for a particular project. In PORT (v. 1.0), factor weights are assigned subjectively based on the discussion between customer and development team. Sensitivity analysis (Srikanth et al., 2005b) was performed to determine an effective and more objective way to allocate factor weights. As will be discussed in Section 4, for PORT (v. 2.0) factor weights

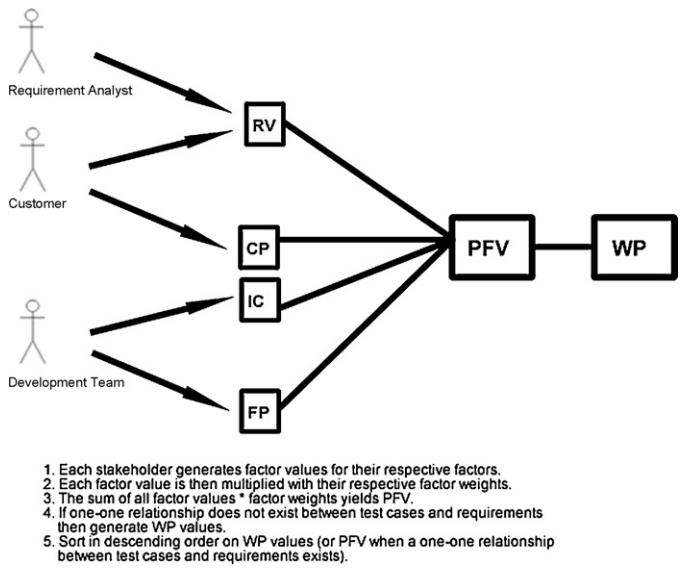


Fig. 2. High level PORT overview.

are assigned proportionally to the mean value of each factor for all the project requirements.

3.3. PORT algorithm

For every requirement, Eq. (3) is used to calculate a Prioritization Factor Value (PFV).

$$\mathrm{PFV}_i = \sum_{j=1}^{4} (\text{Factor Value}_{ij} \star \text{Factor Weight}_j) \tag{3}$$

In Eq. (3), PFV_i represents the Prioritization Factor Value for requirement i, which is the summation of the product of factor value and the assigned factor weight for each of the factors. Factor Value_{ij} represents the value for factor j for requirement i, and Factor Weight_j represents the factor weight for jth factor for a particular product. PFV is a measure of the importance of testing a requirement. A matrix representation of the dependence of the PFV vector P on the value vector V and the weight vector w is shown in Eq. (4) below.

$$\begin{pmatrix} \mathrm{PFV}_1 \\ \cdots \\ \cdots \\ \cdots \\ \mathrm{PFV}_n \end{pmatrix}_{(n*1)}^{P=V_w} = \begin{pmatrix} R_1^{\mathrm{CP}} & R_1^{\mathrm{IC}} & R_1^{\mathrm{RV}} & R_1^{\mathrm{CFP}} \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ R_n^{\mathrm{CP}} & R_n^{\mathrm{CP}} & R_n^{\mathrm{CP}} & R_n^{\mathrm{CP}} \end{pmatrix}_{(n*4)} \begin{pmatrix} w_{\mathrm{CP}} \\ w_{\mathrm{IC}} \\ w_{\mathrm{RV}} \\ w_{\mathrm{FP}} \end{pmatrix}_{(4*1)} \tag{4}$$

The computation of PFV for a requirement is used in computing the Weighted Priority (WP) of its associated test cases. WP of the test case is the PFV contribution of the requirement(s) the test case maps to as shown in Eq. (5). Let there be n total requirements for a product/release, and test case j maps to i requirements. Let X_j denote the set of requirements to which the test case j is mapped.

$$\mathrm{WP}_j = \frac{\sum_{X \in X_j}^{i} \mathrm{PFV}_x}{\sum_{y=1}^{n} \mathrm{PFV}_y} \tag{5}$$

WP_j is an indication of the priority of running a particular test case. The test cases are ordered for execution based on the descending order of WP values such that the test case with the highest WP value is run first and so on. If a one-one ratio exists between test cases and requirements then the value of WP and PFV will be identical.

Suppose we have a scenario with three requirements – R1, R2 and R3 and Factor Weights as 0.2 for CP, 0.3 for IC, 0.25 for FP and 0.25 for FV. Table 2 below illustrates sample Factor Values and computes the PFV for each requirement R1, R2 and R3. Now, suppose we have three test cases T1, T2, T3 that are mapped in a one-one ratio to requirements R1, R2 and R3, respectively. Then utilizing the PORT approach the test cases would be prioritized as T3, T1 and T2.

3.4. PORT validation metrics

Refinement and validation of PORT will proceed via the analysis of the severity of failures detected for a product. For analysis purposes, each failure is assigned a weighted severity value (SV) as follows:

- ***Highly severe (Severity 1)***: Severity 1 is assigned to a failure when a customer can no longer use the product and/or testing must cease until the defect causing the failure is fixed. For Severity 1 failures, we assign a SV of 2^4.
- ***Medium severe (Severity 2)***: Severity 2 is assigned to a failure when there is a work-around for the failure and the product can be used with the work around. For Severity 2 failures, we assign a SV of 2^3.
- ***Less severe (Severity 3)***: Severity 3 is assigned to a failure for which a fix can be done in later versions. For Severity 3 failures, we assign a SV of 2^2.
- ***Least severe (Severity 4)***: Severity 4 is assigned to a failure for which a fix may be done at later versions or not done at all. A SV of 2^1 is assigned to failures with Severity 4.

The PORT validation approach involves computing Total Severity of Failures Detected (TSFD) for the project. TSFD is the summation of severity values (SV) of all failures identified for the product/release. Eq. (6) shows TSFD for a product/release, where t represents total number of failures identified for the product/release.

$$\mathrm{TSFD} = \sum_{l=1}^{t} \mathrm{SV}_l \tag{6}$$

The case studies that utilize TSFD are discussed in Sections 4 and 5.

4. PORT feasibility study

This section discusses our results achieved from a feasibility study of PORT involving four similar student projects (Srikanth, 2004, 2005a; Srikanth and Williams, 2004, 2005; Srikanth et al., 2005a,b). The projects were developed by students in an advanced graduate-level software testing class at North Carolina State University. The average size of the projects was approximately 2500 lines of code (LOC). Since this was an initial release, FP was not considered for this case study. For each project, the students assigned values for RV and IC. The research team (who acted as the customer) assigned values for customer priority (CP). The factor weights were assigned based on discussion between the customer and students.

The students were asked to create an additional faulty version of their project by injecting 20 failures (at least 50% Severity 1 and 2 and the other 50% Severity 3) into their working product. Approximately 50 system test cases were written to test these student projects without any knowledge of the injected failures. The faulty applications were tested to find the injected failures in the application, and the failures found were mapped to their respective requirements. Our goals behind conducting this case study

Table 2
PFV for an example scenario.

Requirement	CP	IC	FP	FV	CP	IC	FP	FV	PFV
R1	5	3	2	4	0.2	0.3	0.25	0.25	3.4
R2	4	4	1	3	0.2	0.3	0.25	0.25	3
R3	3	5	4	5	0.2	0.3	0.25	0.25	4.35

were to determine (1) the effectiveness of the PORT scheme in improving the rate of failure detection; and (2) the contribution of the prioritization factors towards the improved rate of failure detection.

We compared the PORT scheme with a random prioritization strategy for executing tests on the student projects. In this and the industrial case studies, we compare against a random prioritization strategy because informal conversations with practitioners and our own industrial experience indicated that system test most often proceeds without a predetermined strategy for execution. A random approach for test prioritization is utilized by many companies, as evidenced by informal discussions with our industrial partners. After executing each case, the result (Pass/Fail) was noted and the SV was noted for the failure found. The TSFD was computed for all four projects. The test results were then analyzed considering if the test cases were run in the order prescribed by PORT and if they were run in random order. A graph was plotted with the fraction of test suite executed on x-axis, and the percentage of TSFD on y-axis. The rate of detection of failures is computed using a metric called Weighted Percentage of Failures Detected (WPFD), which is the area represented under the curve.

To allow for statistical comparison, 20 random permutations of the test set were generated. The WPFD for PORT was compared against the WPFD for the 20 random permutations. The Wilcoxon Signed Rank Test and the traditional unranked sign test were conducted to investigate the null hypothesis that the WPFD for PORT is no better than that for a randomly chosen prioritization. The Wilcoxon Signed Rank Test and the traditional unranked sign test are both simple nonparametric procedure that makes no assumptions about the distribution of WPFD, used in place of the one sample t-test when the normality assumption is questionable. A correction factor for sample sizes under 30 is used for both statistical tests. For all four teams, we found that the WPFD for PORT is significantly ($p < .01$) higher than random permutation. Also for all four projects, CP contributed the most towards the effectiveness of PORT scheme, followed by IC and RV (Srikanth and Williams, 2005; Srikanth et al., 2005a,b).

A sensitivity analysis was conducted on strategies for assigning factor weights (Srikanth et al., 2005b). For the sensitivity analysis, we considered four possible allocations for factor weights: team-assigned; mean-based; equal weights; and variance-based weight. Team-assigned factor weights are assigned collectively by the customer and the engineering team. Equal weight involves assigning equal weights to all factors, which is analogous to no factor weights. Mean-based weight corresponds with the mean value of each factor for all the project requirements. Variance-based weights are assigned based on the variance of factor values for each project. For all four student projects, we computed the WPFD achieved based on different possible allocations of factor weights. The results show that mean-based approach has yielded the highest rate of failure detection in comparison to the other four approaches; therefore, mean-based weight allocation (as computed in Eqs. (7)–(11)) will be used in industrial case studies.

Suppose w_{CP}, w_{IC}, w_{RV}, and w_{FP} denote the weights for each of the prioritization factors: customer priority, implementation complexity, requirement volatility, and fault proneness, respectively. Suppose a project has n requirements and the factor weight is computed based on the mean of each factor for the project requirements as shown in Eq. (7). Applying bar notation to denote the arithmetic mean, we show the computation of factor weights in Eqs. (8)–(11).

$$\overline{X} = \frac{\sum_{i=1}^{n} x_i}{n} \tag{7}$$

$$w_{CP} = \frac{\overline{CP}}{\overline{CP} + \overline{IC} + \overline{RV} + \overline{FP}} \tag{8}$$

$$w_{IC} = \frac{\overline{IC}}{\overline{CP} + \overline{IC} + \overline{RV} + \overline{FP}} \tag{9}$$

$$w_{RV} = \frac{\overline{RV}}{\overline{CP} + \overline{IC} + \overline{RV} + \overline{FP}} \tag{10}$$

$$w_{FP} = \frac{\overline{FP}}{\overline{CP} + \overline{IC} + \overline{RV} + \overline{FP}} \tag{11}$$

5. Experimental setup and limitations

This section discusses the research methods applied on the three industrial case studies and the limitations of the case studies.

5.1. Research methods

To determine the effectiveness of PORT, we compared PORT with random prioritization for two different test suites for each of the case studies. The first test suite (TS_1) is comprised of the entire set of system test cases for the product, and the second test suite (TS_2) is a subset of the entire set of system test cases. We generated test suite TS_2 by applying the SRET approach (Musa, 1999, 1996), as discussed in Section 2.2, to select a subset of the TS_1 test cases in proportion to the operational profile. In selecting the test suite TS_2, we modified TS_1 such that the number of test cases selected for execution for each requirement is proportional to the operational profile for the requirement. We compared the PORT scheme to random prioritization for both test suites.

The research team performed the following steps:

(1) Computed the factor weights based on mean-based factor weight allocation by applying Eqs. (8)–(11).
(2) Computed PFV for the requirements by applying Eq. (3).
(3) Computed WP for the test cases by applying Eq. (5).
(4) Computed TSFD for the project by applying Eq. (6).
(5) Prioritized the test cases in test suite TS_1 according to the PORT scheme (rank in decreasing order of WP).
(6) Generated 20 randomly selected orderings of test suite TS_1.
(7) Selected a subset of test suite TS_1 based on SRET to generate test suite TS_2.
(8) Prioritized the test cases in test suite TS_2 according to the PORT scheme.
(9) Generated 20 randomly selected orderings of test suite TS_2.

5.2. Threats to validity

We consider the following threats to validity of our case study:

- ***Construct validity*** or establishing measures that reflect the theory under test. Additional system level factors may exist that are

not analyzed in this particular study. We attempt to mitigate this by defining our measures in Section 3 to reflect our four theories, as discussed in Section 1, about utilizing four system-level factors to improve system test efficiency.

- ***Internal validity*** or the occurrence that all the potential factors that might influence the data are controlled except the one under study. Internal validity is a weakness of industrial studies due to the complexity context. Confounding factors may exist that influence the results in the study.
- ***External validity*** or how well the results of the study can be generalized to the world outside the research situation. One may argue that the projects selected do not appropriately reflect all software projects. However, the projects involved in the case studies are highly complex and volatile and used by the customers in the marketplace.
- ***Experimental reliability***, which assesses whether another investigator would get similar results by following the experimental procedures with the same case study. Raw data for the case studies can be found in Srikanth (2005b). The procedures outlined in this paper would enable another researcher to get similar results. As will be discussed in Sections 6.1, we utilized a random selector, which would inhibit direct duplication of our results. However, we ran a sufficient number of trials to feel confident that a replication would produce similar results.

6. Industrial case studies

In this section, we present the results achieved applying PORT post hoc on three industrial case studies at Tekelec and I-Cubed:

- ***Tekelec*** is a leading developer of switching and signaling telecommunications solutions. The Tekelec projects that were analyzed for the case studies provide advanced telecommunication infrastructure which includes signaling technology and network security for new service providers. The product reliability of these products in the field has consistently been 99.99999% with networks with more than 800 nodes. The product has to meet the performance standards expected from the customers in the market place.
- ***I-Cubed*** provides content management for CAD documents. The product enforces the rights assigned to a document such that the end user can only access the protected document in an authorized manner.

The first case study (Tekelec-1) involved analyzing a project that comprised of 75,213 changed lines of code built upon approximately a two million line of code base; 24 modified requirements; and 115 system-level test cases. The second case study (I-Cubed) comprised of 100 thousand lines of code (KLOC), 19 new requirements, and 75 system level test cases. The project provides customers with Digital Rights Protection and version control mechanism. The third case study (Tekelec-2) involved analyzing a second Tekelec project that comprised of 43,701 changed lines of code built upon approximately three and half million lines of code base; 51 new/modified requirements; and 87 system-level test cases. All three projects were written in the Java programming language. The number of test cases selected are on par with typical system level testing practices. In general, a typical system test takes anywhere from 8 h to 4 days to complete. For completeness it is worth nothing that on average 30% of the test cases failed dependent on the run cycle.

For all three case studies, the engineering teams provided the research team with project requirements, test cases, test failures, mapping of failures to test cases, and mappings of test cases to requirements for conducting this post hoc analysis. For these industrial projects, the validation was conducted post hoc by analyzing the test cases executed and defects found by test team when they released the software project. Since the project teams had test cases and defects found during a release cycle, the authors did not consider seeding faults. Examples of such failures include the inability to log into a web based system due to an error in the authentication protocol. Also, the engineering teams provided the factor values for the requirements. RV was assigned based on the number of edits made to the requirement during the development cycle. The edits involved changing the system functionality/feature. CP was assigned based on the system operation needs of customers. IC was assigned post hoc by the engineering team based on the implementation difficulties perceived for the requirements. FP was assigned based on the failures achieved from previous release. FP was applicable only for the two Tekelec case studies and was not applicable for the I-Cubed study as the project was going through initial release and had no prior field data.

Table 3
WPFD for PORT and random for TS_1.

Random versus PORT: TS_1	Tekelec 1	I-Cubed	Tekelec 2
Random permutation 1	53.76	48.72	48.20
Random permutation 2	53.06	61.68	44.60
Random permutation 3	57.70	63.70	46.65
Random permutation 4	47.32	69.46	49.35
Random permutation 5	46.78	54.58	37.95
Random permutation 6	50.06	58.42	44.01
Random permutation 7	47.07	52.10	52.89
Random permutation 8	46.25	46.37	55.44
Random permutation 9	49.11	46.21	57.24
Random permutation 10	47.86	47.30	47.15
Random permutation 11	51.78	52.56	50.29
Random permutation 12	48.29	62.96	53.24
Random permutation 13	43.51	61.46	46.75
Random permutation 14	50.63	56.34	47.75
Random permutation 15	52.22	51.01	48.55
Random permutation 16	43.05	45.94	40.65
Random permutation 17	56.51	46.85	60.59
Random permutation 18	49.56	42.26	48.80
Random permutation 19	53.34	55.34	52.69
Random permutation 20	51.47	41.05	42.95
MEAN WPFD-random TCP	49.96	53.22	48.78
# of times PORT is better than random order	20	20	17
Optimum TCP: WPFD	86.47	96.31	89.00
PORT TCP: WPFD	58.95	84.00	54.00
Sign statistic	20	20	17
Sign test *p*-value	$\leq$0.0001	$\leq$0.0001	$\leq$0.01
Wilcoxon Signed Rank Test *p*-value	$\leq$0.0001	$\leq$0.0001	$\leq$0.001

6.1. Rate of failure detection

The rate of failure detection was determined using WPFD for both test sets, TS_1 and TS_2, for all three industrial studies. Twenty randomly selected orderings of TS_1 and TS_2 test suites were generated to determine the efficacy of the PORT scheme. For both test suites, TS_1 and TS_2, the following hypotheses were tested:

H_0 *PORT TCP is no better than a random permutation when detecting failures.*

H_{alt} *PORT TCP is more effective when detecting failures than a random permutation.*

Table 3 presents the WPFD values for all three case studies for test set TS_1. As will be discussed, PORT improved the rate of failure detection. Additionally, the WPFD for a theoretical optimum prioritized ordering for test sets TS1 for the three case studies are 86.47, 93.31 and 89.00, respectively. The PORT WPFD for the three case studies for test set TS_1 are 58.95, 84.00 and 54.00, which is lower

than the theoretically optimum WPFD value. The optimum WPFD value is achieved when the information about the test case failures already exists, and the test cases are ordered such that those that surface failures with higher weighted severity are executed first. In reality, it is never possible to know which test case will lead to a failure in the field. In our future work, we propose the identification of more factors to further decrease the gap between realized and optimum WPFD.

We apply two non parametric tests, the Wilcoxon Signed Rank Test and the sign test, to determine if there exists a statistically significant difference between PORT and the random permutation when detecting failures. The null hypothesis in a traditional sign test determines whether the difference between the member pairs is positive or negative on chance alone, that is whether the expected value of the difference is equal to 0. The Wilcoxon Signed Rank Test is an extension of a traditional sign test. The Wilcoxon Signed Rank Test ranks the absolute value of each difference and reassigns the observed positive or negative sign, and tests to determine whether the average rank is significantly different from 0. Mathematically, the Wilcoxon Signed Rank Test z-value can be defined as shown in Eq. (12).

$$z = \frac{\bar{r} - 0}{\sqrt{(N+1)(2N+1)/6N}} \tag{12}$$

where $\bar{r}$ is the average rank and N is the total number of observations.

The two statistical tests enable us to reject the null hypothesis and accept the alternate hypothesis that indicates the rate of failure detection for PORT and random permutation is significantly different. PORT does in fact perform better than a random permutation and yields 10–30% better results when detecting failures. In the case of Tekelec-2, only three instances of a random permutation were better than PORT.

Additionally, PORT resulted in improved rate of failure detection than the mean WPFD of 20 random permutations for test suite TS_2 which was generated by applying SRET approach as shown in Table 4. The WPFD for a theoretically optimum prioritized ordering for test set TS_2 are 78.12, 93.95 and 91.00, respectively. The PORT WPFD values for the three case studies for test set TS_2 are 66.14, 88.21 and 61.00.

As before the results of the statistical analysis favored PORT, we reject the null hypothesis and accept that there is a statistically significant difference between the rate of failure detection between PORT and a random permutation. As was the case before, PORT is able to detect on average 10–30% more failures than the random permutation. Tekelec-2 once more exhibited a handful of instances, four, where a random permutation yielded better failure detection than PORT.

In summary, we find statistically significant results in favor of the PORT scheme in comparison to random for both test sets TS_1 and TS_2 ($p < 0.01$) for all three case studies. Figs. 3–5 shows the graphical representation of the rate of failure detection for PORT and mean WPFD for 20 permutations for test sets, TS_1 and TS_2, for all three industrial case studies.

As shown, the rate of failure detection with PORT is higher than the mean rate of failure detection for 20 random permutations. The results show that

- For the Tekelec-1 study, WPFD for PORT (58.95) was higher than the mean WPFD (49.96) for 20 random permutations. Using the PORT scheme, 100% of the failures were surfaced after executing 65% of the test suite whereby the random ordering surfaces 100% of the failures after executing 100% of the test suite.
- For I-Cubed project, the WPFD for PORT (82.49) was higher than the mean WPFD (53.22) for 20 random permutations. The PORT scheme led to surfacing 100% of the failures after executing 70% of the test suite where by the random ordering surfaces 100% of the failures after executing 92% of the test suite.
- Finally, for Tekelec-2, the WPFD for PORT (54.00) was higher than mean WPFD (48.78) for 20 random permutations.

Table 4
WPFD for PORT+SRET and SRET alone.

Random versus PORT for TS_2	Tekelec-1	I-Cubed	Tekelec-2
Random permutation 1	52.48	41.26	45.33
Random permutation 2	49.95	54.82	50.66
Random permutation 3	51.42	55.26	77.33
Random permutation 4	52.29	44.73	28.00
Random permutation 5	52.52	56.82	48.66
Random permutation 6	44.14	57.60	25.33
Random permutation 7	44.58	52.13	54.00
Random permutation 8	46.52	38.91	60.00
Random permutation 9	45.61	42.82	66.00
Random permutation 10	48.24	51.52	72.66
Random permutation 11	56.43	46.56	34.00
Random permutation 12	43.28	67.78	62.66
Random permutation 13	47.95	74.73	26.66
Random permutation 14	54.85	53.34	60.66
Random permutation 15	55.52	90.82	36.66
Random permutation 16	50.39	41.69	55.33
Random permutation 17	49.23	49.23	58.00
Random permutation 18	59.02	46.82	53.33
Random permutation 19	50.85	51.86	47.33
Random permutation 20	48.09	66.30	32.00
MEAN WPFD-random TCP	50.2	54.63	49.73
# of times PORT is better than random order	20	19	16
Optimum TCP: WPFD	78.12	93.95	91.00
PORT TCP: WPFD	66.14	88.21	61.00
Sign statistic	20	19	16
Sign test p-value	$\leq$0.0001	$\leq$0.0001	$\leq$0.01
Wilcoxon Signed Rank Test p-value	$\leq$0.0001	$\leq$0.0001	$\leq$0.005

6.2. Contribution of PORT factors

As discussed, this research aims to identify the minimal set of prioritization factors for an efficient and effective system-level TCP scheme to reduce the overhead of applying the scheme. The analysis was extended to determine if the PORT scheme can lead to improved efficiency without the incorporation of the any one of the least effective factors in our studies to date. The results achieved were compared to the results achieved by excluding each of the four factors individually. The goal is to determine if the rate of failure detection can be as effective or nearly as effective without the inclusion of any these four factors to save the burden of tracking this measure. The results are presented in Table 5. WPFD reduced the most when CP was excluded. These results support our theory T1 that *customer-assigned priority of requirements is an important factor to consider in improving system test efficiency*.

As shown in Table 5, WPFD reduced when IC was removed for two of three case studies. These results support our theory T2 that *developer perceived implementation complexity is an important factor to consider in improving system test efficiency*.

Table 5
Contribution of the PORT factors.

	Tekelec-1	I-Cubed	Tekelec-2
All four factors (RV, IC, CP and FP)	58.95	84.00	54.00
Excluding CP (IC, FP and RV)	**54.00**	**81.00**	**49.00**
Excluding IC (CP, RV and FP)	**56.00**	**79.00**	54.00
Excluding FP (CP, RV and IC)	**57.00**	Not applicable	56.00
Excluding RV (IC, CP and FP)	58.00	84.00	55.00

Bold indicates a reduction in WPFD when the factor is removed.

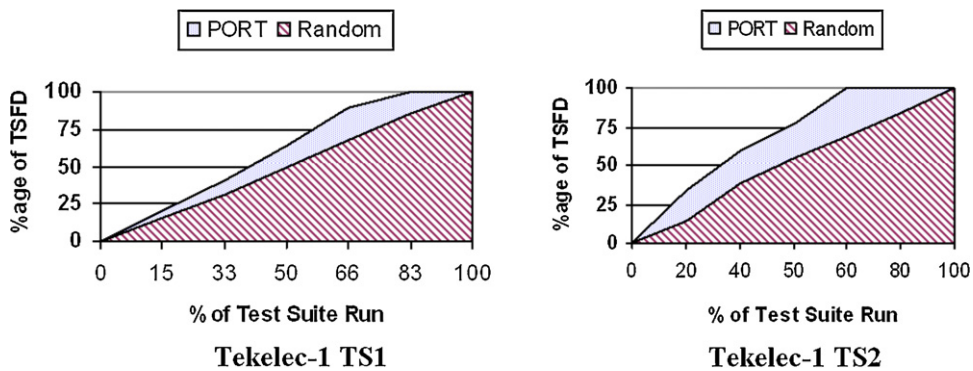


Fig. 3. Tekelec-1.

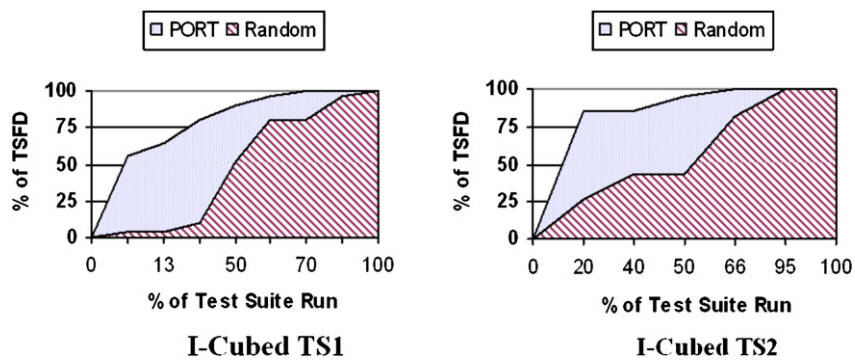


Fig. 4. I-Cubed.

FP was applicable only for the two Tekelec projects. Since the I-Cubed project was going through initial release, FP was not applicable. FP was a contributor for the Tekelec-1 case study as exclusion of FP from the scheme resulted in reduction in WPFD value. This supports our theory T3 that *fault proneness is an important factor to consider in improving system test efficiency*. However, for Tekelec-2 case study, FP was not a contributor as exclusion of FP resulted in improved WPFD value in comparison to the WPFD value achieved via PORT. The engineering team indicated that some of the fault prone requirements comprised of high customer value requirements and underwent thorough extensive in-house system testing, which is believed to reflect upon the negative impact of FP towards improved rate of failure detection for this project.

RV was a negative contributor for Tekelec-2 project, and had little or no impact on the Tekelec-1 and I-Cubed projects. Removal of RV from the PORT scheme had either negative impact or little to no impact on the WPFD values. These results do *not* support our theory T4 that *requirements volatility is an important factor to consider in improving system test efficiency*. However, further study is needed prior to removing RV from PORT since prior research, as discussed in Section 3.1.4 would indicate RV would be important.

The mean and variance for the factor values for all three case studies were computed and the results are shown in Table 4. The mean RV value for project requirements for the Tekelec-1 project is 1.5 with a variance of 1.0; the mean RV value for project requirements for the Tekelec-2 project is 1.7 with a variance of 1.7. For these two projects, the requirements were stable which is likely to have led to RV not being an important factor towards improving test efficiency. For the I-Cubed project, the mean RV value for all project requirements is 4.6 with a variance of 2.9. All requirements for this project were new as it was going through initial release. Thus, RV did not have any impact on the rate of failure detection.

However, for both Tekelec projects, the CP was the high with a mean CP value of 6.8 and 5.9 with a variance of 2.2 and 9.5,

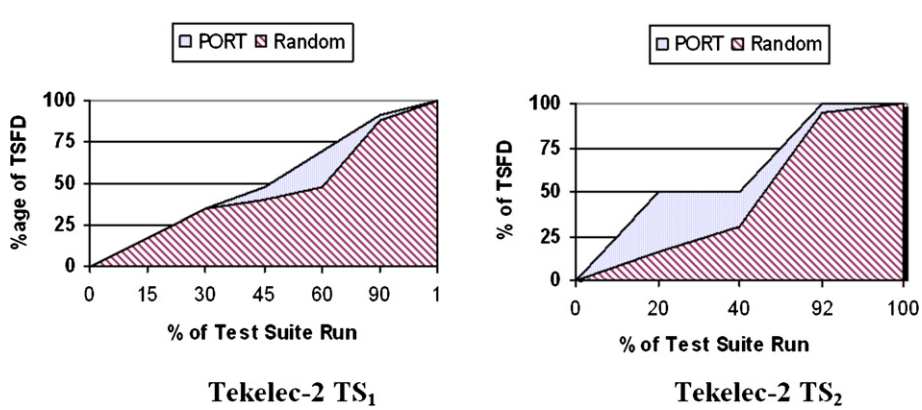


Fig. 5. Tekelec-2.

Table 6
Mean and variance of factor values.

	RV	IC	*CP*	FP
Tekelec-1: Mean	1.5	3.0	6.8	2.5
Tekelec-1: Variance	1.0	1.6	2.2	1.1
I-Cubed: Mean	4.6	6.7	8.6	N/A
I-Cubed: Variance	2.9	2.6	1.9	N/A
Tekelec-2: Mean	1.7	3.2	5.9	3.4
Tekelec-2: Variance	1.7	4.6	9.5	4.6

respectively, indicating that CP was an important factor for these projects. IC was the highest contributing factor for I-Cubed project with a mean and variance 6.7 and 2.6, respectively. The second highest contributor was CP with a mean and variance of 8.6 and 1.9, respectively (Table 6).

7. Summary

Software testing is an expensive process; system testing is the most expensive and especially complex of all types of testing (Harrold, 2000). Our goal was to develop and validate a value-based system-level test case prioritization scheme for improving the rate of failure detection. The PORT scheme has been applied in four case studies (one academic, three industrial). Four prioritization factors are incorporated in our prioritization scheme: customer-assigned priority (CP), developer-perceived implementation complexity (IC), fault proneness (FP), and requirements volatility (RV).

PORT efficacy was determined by comparing the rate of failure detection for PORT to a random prioritization for two sets of test suites: entire test set (TS_1); and subset of entire set (TS_2) by applying SRET approach with PORT. The rate of failure detection for the 20 random permutations was compared with that of the PORT scheme for both test sets. Using two non parametric tests, the Wilcoxon Signed Rank Test and the traditional sign test, the rate of failure detection for the PORT scheme was found to be significantly better than a random ordering of the test sets, based on 20 such permutations for all three case studies. Additionally, the comparison of results for test set TS_1 with test set TS_2 show that rate of failure detection for PORT scheme and the mean WPFD value for 20 random permutations are higher for test set TS_2, which is generated via SRET approach. These results suggest that PORT combined with SRET yielded higher rate of failure detection than PORT alone.

Via this research the following four theories were being evolved:

T1. ***Customer-assigned priority*** *of requirements is an important factor to consider in improving system test efficiency.*

T2. ***Developer-perceived implementation complexity*** *is an important factor to consider in improving system test efficiency.*

T3. ***Fault proneness*** *of requirements is an important factor to consider in improving system test efficiency.*

T4. ***Requirements volatility*** *is an important factor to consider in improving system test efficiency.*

The contribution of each of the prioritization factors towards the test efficiency was examined via our case studies. Our results show the following:

1. Customer priority was a significant contributor for all three case studies thus *supporting our theory T1*. Removal of CP from the PORT scheme for all three case studies resulted in reduced rate of failure detection. These results suggest that the PORT scheme could improve the effectiveness of testing activities by focusing on functionalities that are of high value to the customer.
2. IC was a significant contributor for Tekelec-1 and I-Cubed project thus *supporting theory T2*. Although, IC did not have an impact on the Tekelec-2 project, it was a highest contributor for the I-Cubed project and an important factor for the Tekelec-1 project.
3. FP was applicable only for the two Tekelec case studies and not applicable for the I-Cubed project as the project was going through initial release. FP was a significant contributor for Tekelec-1 project as exclusion of FP from the PORT scheme resulted in reduced rate of failure detection thus *supporting theory T3*. However, FP had a negative impact on Tekelec-2 project. The exclusion of FP also resulted in improved rate of failure detection, suggesting that FP has not been a contributor towards test efficiency for this project. This is believed to be due to the extensive testing conducted on Tekelec-2 project in prior releases on the fault prone requirements. The engineering team indicated that some of the fault prone requirements comprised of high customer value requirements and underwent thorough in-house system testing, which is believed to reflect upon the negative impact of FP towards improved rate of failure detection for this project. Although FP was not a contributor for one of the Tekelec-2 project, it was an important contributor for Tekelec-1 project.
4. Requirements volatility had little to none or negative impact on the rate of failure detection for all three case studies. The results are *not supportive of theory T4*. For all three case studies, exclusion of RV had negative impact or minimal to no impact on the rate of failure detection in comparison to PORT scheme, which is reflective of the fact that the RV was not a contributor. These results are not consistent with prior research, as discussed in Section 3.1.4. Therefore, the contradictory results warrant analysis in additional case studies to further evolve and test theory T4.

The results support theories T1, T2 and T3 and show that customer-assigned priority for requirements, developer perceived implementation complexity and fault proneness of requirements are factors that can improve rate of failure detection when considered during system test phase. Additionally, the act of writing concrete test cases immediately after requirements specification can lead to the identification of ambiguous and unclear requirements, allowing requirements errors to be identified and rectified earlier. The PORT scheme allows the engineering team to monitor the requirements covered in system test; the ability to monitor requirements covered in system test is believed to be one of the challenges faced by the industry (Elbaum et al., 2001). By focusing on customer-assigned priority, we aim to identify requirements which would increase customer value, thus improving the overall business value provided to the customers.

Acknowledgment

We would like to thank Richard Laurent at Tekelec and Grant Williard at I-Cubed for helping us with the industrial case studies.

References

Amland, S., 1999. Risk based testing and metrics. In: 5th International Conference EuroSTAR '99, Barcelona, Spain, pp. 1–20.
Basanieri, F., Betolino, A., Marchetti, E., 2002. The Cow_Suite approach to planning and deriving test suites in UML projects. In: Fifth International Conference on the Unified Modeling Language – the Language and its Applications UML 2002, Dresden, Germany, September, pp. 383–397.
Beizer, B., 1990. Software Testing Techniques. Van Nostrand Reinhold, New York, NY.
Boehm, B., Huang, L., 2003. Value-based software engineering: a case study. IEEE Computer 36 (March (3)), 33–41.
Boehm, B., 2003. Value-based software engineering. ACM Software Engineering Notes 28 (March (2)), 1–12.
Craig, R., Jaskiel, S., 2002. Systematic Software Testing. Artech House Publishers, Norwood, MA.

Donovan, D.C., Dislis, C., Murphy, R., Unger, S., Keanneally, C., Young, J., Sheehan, L., 2001. Incorporating software reliability engineering into the test process for an extensive GUI-based network system. In: International Symposium on Software Reliability Engineering, Hong Kong, pp. 44–53.
Elbaum, S., Malishevsky, A., Rothermel, G., 2001. Incorporating varying test costs and fault severities into test case prioritization. In: 23rd International Conference on Software Engineering, Ontario, Canada, May, pp. 329–338.
Elbaum, S., Malishevsky, A., Rothermel, G., 2002. Test case prioritization: a family of empirical studies. IEEE Transactions on Software Engineering 28 (February (2)), 159–182.
Fraser, G., Wotawa, F., 2007. Test-case prioritization with model-checkers. In: Proceedings of the 25th Conference on IASTED International Multi-Conference: Software Engineering (SE'07), pp. 267–272.
Harrold, M., 2000. Testing: a roadmap. In: International Conference on Software Engineering, Limerick, Ireland, pp. 61–72.
House, D., Newman, W., 1989. Testing large software products. ACM Software Engineering Notes 14 (2), 71–78.
IEEE, 1990. IEEE Standard 610.12-1990, IEEE Standard Glossary of Software Engineering Terminology.
Jones, C., 1996. Software challenges: strategies for managing requirements creep. IEEE Computer 29 (June (6)), 92–94.
Karlsson, J., Ryan, K., 1997. A cost-value approach for prioritizing requirements. IEEE Software 14 (September–October (5)), 67–74.
Korel, B., Koutsogiannakis, G., 2009. Experimental comparison of code-based and model-based test prioritization. In: Proceedings of the IEEE International Conference on Software Testing, Verification, and Validation Workshops (ICSTW '09), pp. 77–84.
Korel, B., Tahat, L., Vaysburg, B., 2002. Model based regression test reduction using dependence analysis. In: Proc. IEEE International Conf. on Software Maintenance, pp. 214–223.
Korel, B., Tahat, L.H., Harman, M., 2005. Test prioritization using system models. In: Proc. 21st IEEE Int'l Conf. Software Maintenance (ICSM '05), pp. 559–568.
Korel, B., Koutsogiannakis, G., Tahat, L.H., 2007. Model-based test prioritization heuristic methods and their evaluation. In: Proceedings of the 3rd International Workshop on Advances in Model-Based Testing (A-MOST '07), pp. 34–43.
Malaiya, Y.K., Denton, J., 1999. Requirements volatility and defect density. In: 10th Intl' Symposium on Software Reliability Engineering, Boca Ratan, Florida, November, pp. 285–298.
Mogyorodi, G., 2001. Requirements-based testing: an overview. In: 39th International Conference and Exhibition on Technology of Object-Oriented Languages and Systems, Santa Barbara, California, August, pp. 286–295.
Munson, J.C., Elbaum, S., 1999. Software reliability as a function of user execution patterns and practice. In: 32nd Annual Hawaii International Conference of System Sciences, Maui, HI, pp. 255–285.
Musa, J., 1996. Software-reliability-engineered testing. IEEE Computer 29 (November (11)), 61–68.
Musa, J., 1999. Software Reliability Engineering. McGraw-Hill, New York, NY.
O'Neal, J., Carver, D., 2001. Analyzing the impact of changing requirements. In: IEEE International Conference on Software Maintenance, Los Alamitos, California, pp. 190–195.
Ostrand, T., Weyuker, E., Bell, R., 2004. Where the bugs are. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis, Boston, MA, July, pp. 86–96.
Park, J.-W., Port, D., Boehm, B., 1999. Supporting distributed collaborative prioritization. In: Software Engineering Conference, APSEC '99, pp. 560–563.
Rothermel, G., Untch, R., Chu, C., Harrold, M., 1999. Test case prioritization: an empirical study. In: International Conference on Software Maintenance, Oxford, UK, September, pp. 179–188.
Rothermel, G., Untch, R., Chu, C., Harrold, M., 2001. Test case prioritization. IEEE Transactions on Software Engineering 27 (October (10)), 929–948.
Shull, F., Basili, V., Boehm, B., Brown, W., Costa, P., Lindvall, M., Port, D., Rus, I., Tesoriero, R., Zelkowitz, M., 2002. What we learned about fighting defects. In: IEEE Symposium on Software Metrics, Ottawa, Canada, June, pp. 249–258.
Srikanth, H., Williams, L., 2004. Requirements Based Test Case Prioritization, Poster Session and Ph.D. Forum in Grace Hopper Celebration of Women in Computing, Chicago, Illinois, 2004.
Srikanth, H., Williams, L., 2005. On economic benefits of system level test case prioritization. In: Seventh International Workshop on Economics-Driven Software Engineering Research (EDSER) at International Conference on Software Engineering, St. Louis, MO.
Srikanth, H., Williams, L., Osborne, J., 2005a. System Test case prioritization of new and regression tests. In: International Symposium of Empirical Software Engineering, Noosa Heads, Australia, pp. 64–73.
Srikanth, H., Williams, L., Osborne, J., 2005b. Towards the prioritization of system test cases. North Carolina State University TR-2005-44.
Srikanth, H., Cohen, M., Qu, X., 2009. Reducing field failures in system configurable software: cost-based prioritization. In: Int'l Symposium on Software Reliability Engg, November, pp. 61–70.
Srikanth, H., 2004. Requirements based test case prioritization. In: Student Research Forum in 12th ACM SIGSOFT Int'l Symposium on the Foundations of Software Engineering, Newport Beach, California.
Srikanth, H., 2005a. Requirements-based test case prioritization. In: Doctoral Symposium in International Conference of Software Engineering, St. Louis, MO.
Srikanth, H., 2005b. Value-Driven System-Level Test Case Prioritization, Doctoral Dissertation. North Carolina State University, Department of Computer Science.
Standish.Group, 1995. CHAOS. http://www.standishgroup.com/.
Tahat, L., Vaysburg, B., Korel, B., Bader, A., 2001. Requirement-based automated black-box test generation. In: 25th Annual International Computer Software and Applications Conference, Chicago, Illinois, pp. 489–495.
Vouk, M., Rivers, A.T., 2003. Construction of reliable software in resource-constrained environments. In: Blischke, W.R., Murphy, D.N.P. (Eds.), Case Studies in Reliability and Maintenance. John Wiley and Sons, Hoboken, NJ, pp. 205–231.
Wong, E., Horgan, J., Syring, M., Zage, W., Zage, D., 2000. Applying design metrics to predict fault-proneness: a case study on a large-scale software system. Software Practice and Experience 30, 1587–1608.

Dr. Hema Srikanth joined IBM after receiving her doctorate in Computer Science from North Carolina State University, and MBA from Eberly College of Business. At her current role as test architect, she is responsible for test strategy and customer support strategy for IBM's first SaaS offering. Prior to joining the development team as a test architect, she was a quality champion for IBM's Lotus product offerings in Software Group. As a quality champion, she led the test optimization effort for her division to consistently improve test efficiency. Her current research focuses on software quality, software reliability and testing, software metrics, and in advanced analytics.

Sean Banerjee is a PhD student at West Virginia University conducing research on evaluating software reliability in service based systems.